The ngspice `optimize` command

A friendly user manual for the built-in parameter optimizer

Ngspice + OpenVAF Enhancements project

July 2026

Contents

The ngspice **`optimize`** command — a friendly user manual

A built-in parameter optimizer for ngspice (Enhancement-130, with least-squares curve fitting added in Enhancement-143, symbolic `.param` tuning in Enhancement-144, and `.model`-card parameter tuning in Enhancement-145).

This guide explains how to use the `optimize` command from scratch. You do not need any background in optimization or numerical methods — if you can write a small SPICE netlist and run it, you can use the optimizer. We build up from the simplest possible circuit and end with a few genuinely useful design tasks, with pictures at every step.

---

1. What problem does it solve?

When you design a circuit you usually know what you want the circuit to do — a gain of exactly 0.5, a filter that rolls off at 1 kHz, an output that reaches 0.9 V after 1 ms — but you don't know the exact component values that achieve it. The normal way to find them is trial and error: change a resistor, run the simulation, look at the result, change it again, and repeat until it's close enough.

The `optimize` command does that loop for you, automatically and quickly. You tell it:

- which knobs to turn (which component values it may change, and their allowed range),
- what to run (a normal ngspice analysis: `op`, `ac …`, `tran …`),

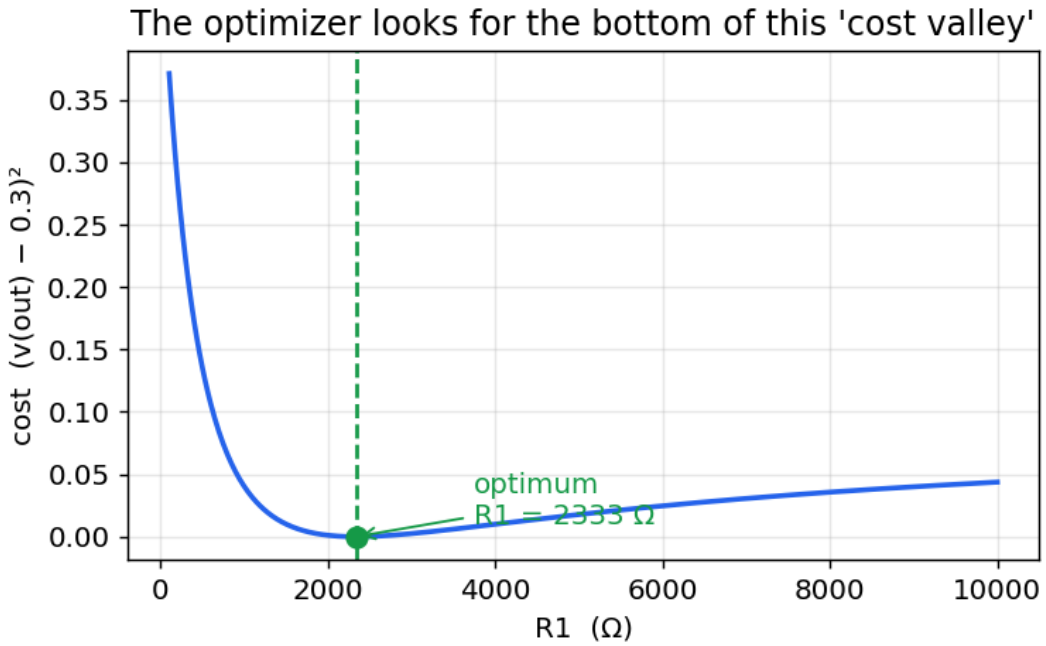
- what ”good” means (a small mathematical expression that is *zero* when the circuit does exactly what you want),

and it turns the knobs for you until the circuit is as close to your goal as possible.

The mental picture

Think of the thing you are trying to make small — how far the circuit is from your goal — as the height of a landscape. Turning a knob moves you east–west; the ”badness” of the result is how high up you are. Your goal sits at the very bottom of a valley. The optimizer is like a hiker who can't see the whole map but can feel which way is downhill, and keeps stepping down until they reach the bottom.

Here is that landscape for our very first example (turning one resistor `R1`, trying to make the output 0.3 V). The optimizer's job is simply to find the bottom of this valley:



The number plotted vertically is called the cost (or *objective*): it is `0` when the circuit is perfect and grows as the circuit gets worse. You define the cost; the optimizer only ever tries to make it small.

---

2. The command at a glance

You run `optimize` inside a `.control … .endc` block, after your circuit is loaded:

```
optimize (-param|-mparam|-dparam) <name> <init> <lo> <hi>   [...]
         -analysis <command ...>
         ( -minimize <expression ...>                          (one goal)
           | -target <expr> <value> [<weight>]  [-target ...] ) (fit several)
         [-method nm|lm|pso|de|sa] [-maxiter <N>] [-tol <T>] [-seed <S>] [-verbose]
```

Part	Meaning
`-param name init lo hi`	A knob to turn. `name` is a device (like `R1`, `C1`) or a device instance parameter (`@m1[w]`). `init` is where to start, `lo`/`hi` are the smallest/largest values allowed. Repeat for each knob (up to 128 — see §12).

Part	Meaning
`-mparam name init lo hi`	Like `-param`, but `name` is a **`.model`**-card parameter, written `@<model>[<param>]` (e.g. `@dmod[is]`). See §9. Mixes freely with the others.
`-dparam name init lo hi`	Like `-param`, but `name` is a symbolic netlist **`.param`** (e.g. the `w` in `.param w=1u`, or a name used in an expression like `R1={500*k}`). See §9. Mixes freely with the others.
`-analysis <cmd>`	The simulation to run every time it turns the knobs — an ordinary ngspice command such as `op`, `ac dec 20 1 1meg`, or `tran 1u 1m`. Give several to combine analyses in one fit (see §8).
`-minimize <expr>`	The cost, for a single goal. Any ngspice expression over the results that should be zero when the circuit is perfect. A very common shape is `(something - target)^2`.
`-target <expr> <val> [<w>]`	A measurement to fit (§8). Repeat to fit many at once; the optimizer minimizes the sum of squared residuals `w·(expr - val)`. Use `-target` *or* `-minimize`, not both.
`-method nm|lm|pso|de|sa`	(optional) pick the search. Local (go downhill from the start): `nm` (Nelder-Mead), `lm` (Levenberg-Marquardt). Global (explore, to escape local minima — §12): `pso` (particle swarm), `de` (differential evolution), `sa` (simulated annealing). Default: a `-target` fit uses `lm`, a `-minimize` goal uses `nm`.
`-maxiter N`	(optional) stop after at most `N` steps. Default `100`.
`-tol T`	(optional) stop when the cost stops improving by more than `T`. Default `1e-6`.
`-seed S`	(optional) fixed random seed for the global methods (`pso`/`de`/`sa`) so a run is exactly repeatable.
`-swarmsize N`	(optional) population size for `pso`/`de` (default auto-scales with the number of knobs).
`-verbose`	(optional) print the cost after every step so you can watch it fall.

A couple of friendly details so you don't trip up:

- `-analysis` and `-minimize` gobble up all the words after them until the next `-word` flag, so you can write `-analysis ac dec 20 1 1meg` or `-minimize (v(out)-0.3)^2` without quotes.
- A negative lower bound like `-5` is understood as a number, not a flag (flags start with `-` followed by a *letter*).
- The hundreds of little simulations the optimizer runs are silent by default, so your screen isn't flooded. Add `-verbose` if you want to watch progress.

---

3. Example 1 — the simplest possible case (one knob)

A voltage divider: a 1 V source drives two resistors in series, and we measure the voltage at their midpoint, `out`. Basic circuit theory says `v(out) = R2 / (R1 + R2)`. With `R2 = 1 k`Ω, if we want `v(out) = 0.3 V`, the exact answer is `R1 = 2333.3` Ω. Let's pretend we don't know that and let the optimizer find it.

```
Voltage divider: tune R1 so v(out) = 0.3 V
V1 in  0   dc 1
```

```
R1 in  out 1k
R2 out 0   1k

.control
optimize -param R1 1k 100 10k -analysis op -minimize (v(out)-0.3)^2
print v(out)
.endc
.end
```

Reading the `optimize` line in plain English: *"turn R1 (start at 1 kΩ, keep it between 100 Ω and 10 kΩ); each time, run an operating-point (op) analysis; and try to make `(v(out) − 0.3)`^{2} as small as possible."* That expression is zero exactly when `v(out)` equals `0.3`, which is what we want.

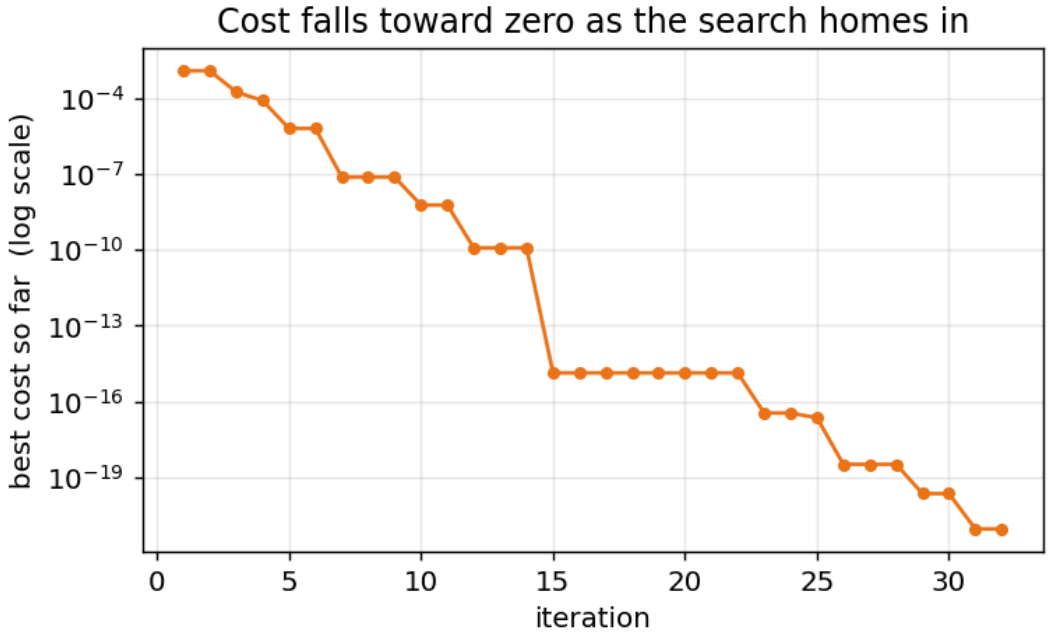
Running it prints:

```
optimize: 1 parameter, analysis 'op', minimizing '(v(out)-0.3)^2'
optimize: converged, objective = 8.99997e-22 after 67 evaluations
    r1 = 2333.33
v(out) = 3.000000e-01
```

The optimizer found `R1 = 2333.33` Ω — the exact textbook answer — and left the circuit sitting at that value, so `v(out)` is now `0.3 V`. The `objective = 9e-22` is essentially zero (the cost at the bottom of the valley).

Watching it work

Add `-verbose` and the optimizer prints the best cost after each step. Plotting those numbers shows the search closing in on the answer — the cost plunges toward zero:



That is the whole idea in one picture: start somewhere, keep stepping downhill, stop when you can't do better.

---

4. Example 2 — designing a filter's gain (AC analysis)

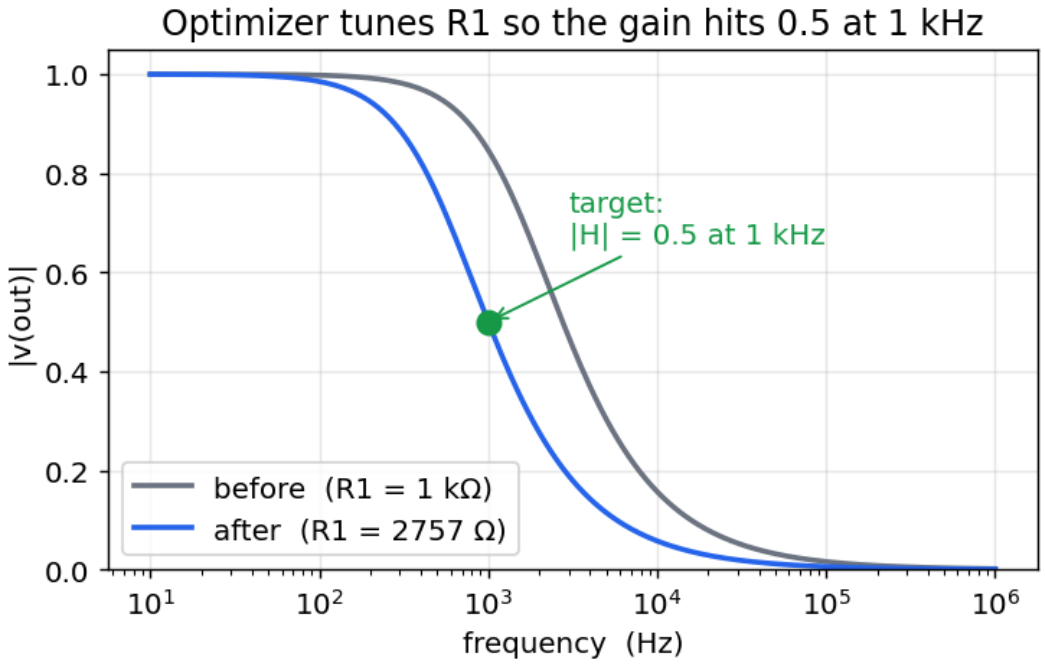
Now something a circuit designer actually does. An RC low-pass filter: a resistor `R1` into a capacitor `C1`. Its gain falls with frequency. Suppose we want the gain to be exactly 0.5 at 1 kHz, and we're allowed to choose `R1` (with `C1 = 100 nF` fixed).

We run an AC analysis at just the one frequency of interest (1 kHz) and ask that the magnitude of the output there be 0.5:

```
RC low-pass: tune R1 so |gain| = 0.5 at 1 kHz
V1 in  0   ac 1
R1 in  out 1k
C1 out 0   100n

.control
optimize -param R1 1k 100 100k -analysis ac lin 1 1k 1k -minimize
 ↪  (mag(v(out))-0.5)^2
.endc
.end
```

`ac lin 1 1k 1k` runs the AC analysis at a single point, 1 kHz. `mag(v(out))` is the gain magnitude there, and we drive $(\texttt{mag(v(out))} - \texttt{0.5})^2$ to zero. The optimizer finds `R1 = 2756.6` Ω (which is the exact value where $\texttt{1/}\sqrt{\texttt{(1+(2πfRC)}^2\texttt{)}} = \texttt{0.5}$).

Here is the filter's response before (the starting `R1 = 1` kΩ) and after the optimizer tuned it — the curve slides over until it passes exactly through the target point:



The optimizer only ever looked at the single point at 1 kHz, but because it moved `R1` the whole curve shifted with it, landing the 1 kHz gain right on 0.5.

---

5. Example 3 — two knobs at once

Real designs have several free values. The optimizer handles many parameters together — just add more `-param` flags. Here we tune both resistors of a divider so that two things are true at once:

- the output is `0.4 V`, and
- the total resistance is `5 kΩ` (so the current drawn from the 1 V source is `0.2 mA`).

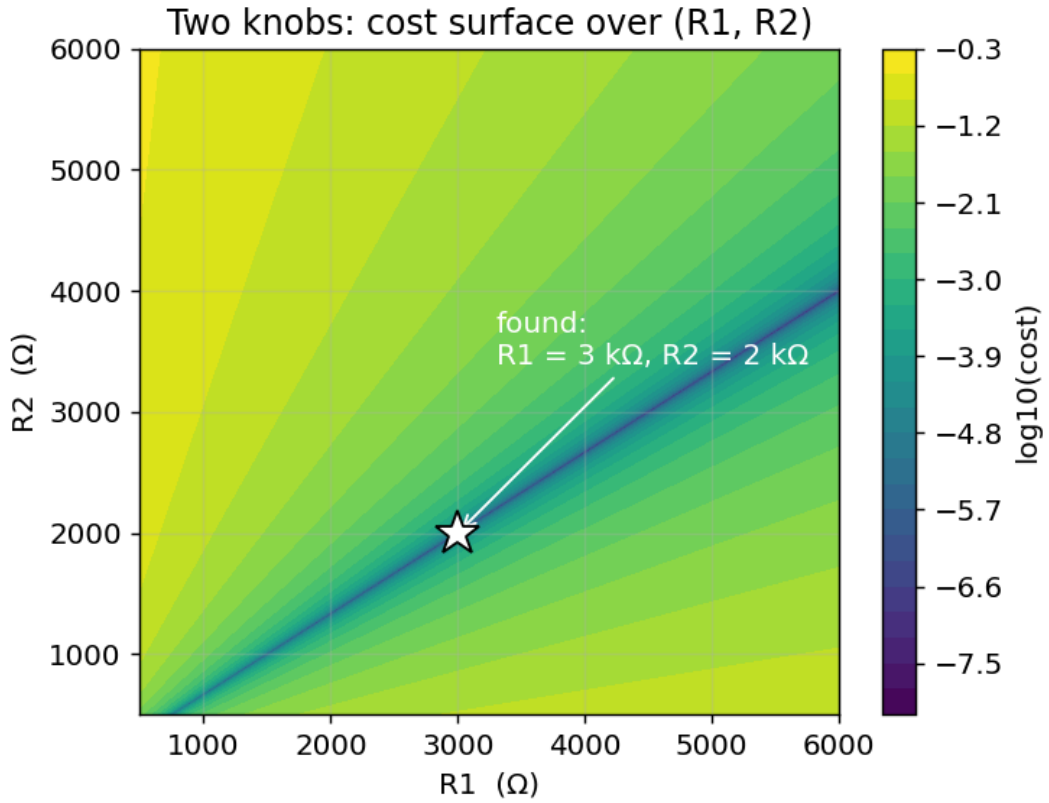
There is exactly one solution: `R1 = 3 kΩ`, `R2 = 2 kΩ`. We combine the two goals by adding their costs — the total is zero only when *both* are satisfied:

```
Two-parameter divider design
V1 in  0   dc 1
R1 in  out 1k
R2 out 0   1k

.control
optimize -param R1 1k 100 10k -param R2 1k 100 10k
+        -analysis op
+        -minimize (v(out)-0.4)^2 + (abs(i(v1))-0.2m)^2
+        -maxiter 400 -tol 1e-15
print v(out) i(v1)
.endc
.end
```

(Note the + at the start of the continuation lines — that is ngspice's normal way of splitting one long line.) The optimizer reports:

```
optimize: converged, objective = 4e-26 after 192 evaluations
    r1 = 3000
    r2 = 2000
v(out) = 4.000000e-01
i(v1) = -2.00000e-04
```

Exactly `R1 = 3 kΩ`, `R2 = 2 kΩ`. Now the cost is a surface over two knobs instead of a valley over one. The bright colors are high cost, the dark diagonal streak is the ”trench” of good solutions, and the star marks the single point where both goals are met and that the optimizer walked to:



This is why the optimizer is worth having: even with two coupled requirements pulling in different directions, it finds the one combination that satisfies both, from a plain starting guess of 1 kΩ / 1 kΩ.

---

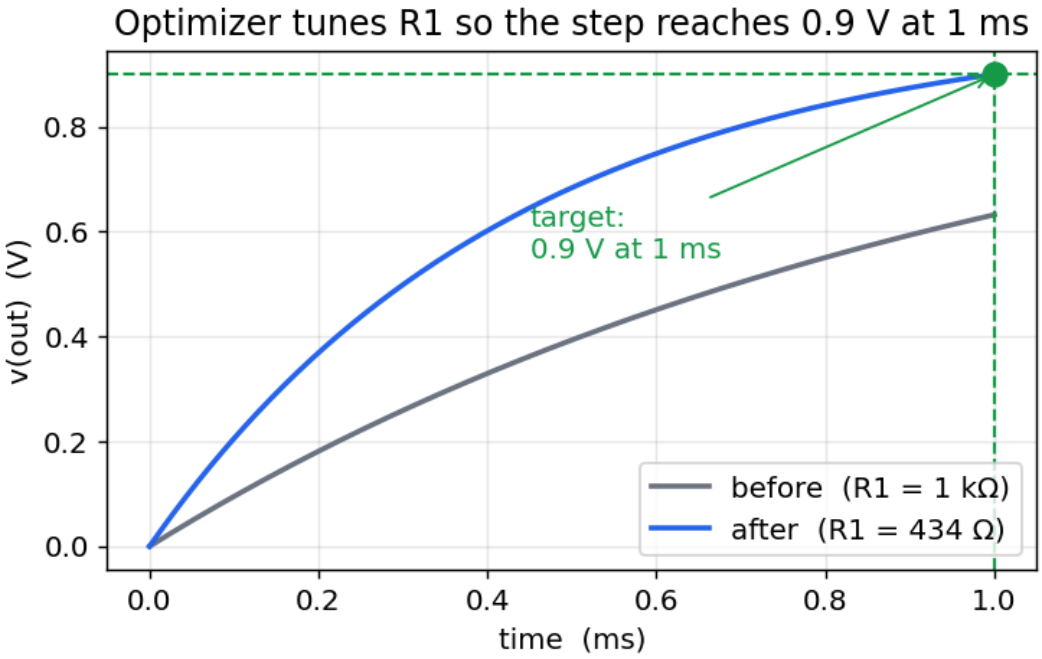
6. Example 4 — a time-domain goal (transient analysis)

Goals don't have to be about voltages or gains — they can be about timing. Take an RC circuit charging from a 1 V step. We want the output to reach 0.9 V exactly 1 ms after the step, by choosing `R1` (with `C1 = 1 µF`).

We run a transient out to 1 ms and ask that the output *at the end of the run* be 0.9 V:

```
RC step: tune R1 so v(out) = 0.9 V at t = 1 ms
V1 in  0   dc 1
R1 in  out 1k
C1 out 0   1u ic=0

.control
optimize -param R1 100 50 5k -analysis tran 2u 1m uic -minimize (v(out)-0.9)^2
.endc
.end
```

Because `tran 2u 1m` stops at 1 ms, the *last* value of `v(out)` is the voltage at 1 ms, and that is what the cost compares to 0.9. The optimizer finds `R1 = 434.3 Ω` (the exact value where `1 − e^(−1ms/RC) = 0.9`). The step response before and after:



The slow starting curve (1 kΩ) only reaches ~0.63 V by 1 ms; after tuning, the faster curve passes right through the target dot at 0.9 V / 1 ms.

---

7. Optimizing OSDI / Verilog-A devices

Everything so far used ngspice's built-in components, but the real strength of this build is compiled Verilog-A models (OSDI): you write a device in Verilog-A, compile it to a `.osdi` file with `openvaf-r`, and load it with `pre_osdi`. The optimizer tunes those devices in exactly the same way — you just point `-param` at an OSDI *instance parameter*.

One rule to remember. For the optimizer to change a Verilog-A parameter, that parameter must be an instance parameter, so mark it with `(*type="instance"*)` in the model. (A plain `parameter` is a *model* parameter — shared by every instance and not reachable by the per-instance `alter` the optimizer uses.) You then refer to it as `@<instance>[<param>]`.

A Verilog-A resistor

Here is a one-line resistor whose resistance `r` is an instance parameter:

```
// optres.va
`include "disciplines.vams"
module optres(p, n);
    inout p, n;
    electrical p, n;
    (*type="instance"*) parameter real r = 1000.0 from (0:inf);
    analog
        I(p, n) <+ V(p, n) / r;
endmodule
```

Compile it with `openvaf-r optres.va -o optres.osdi` and drop it into the divider from Example 1. The only change is that `R1` is now the Verilog-A device `N1`, and the knob is `@n1[r]`:

```
OSDI resistor divider: tune the Verilog-A resistor
V1 in 0 dc 1
```

```
N1 in out rmod
R2 out 0 1k
.model rmod optres r=1k

.control
pre_osdi optres.osdi
optimize -param @n1[r] 1k 100 10k -analysis op -minimize (v(out)-0.3)^2
.endc
.end
```

The optimizer finds `@n1[r] = 2333.33` Ω — the same answer as Example 1, now for a compiled device.

Parameter extraction: fitting a diode

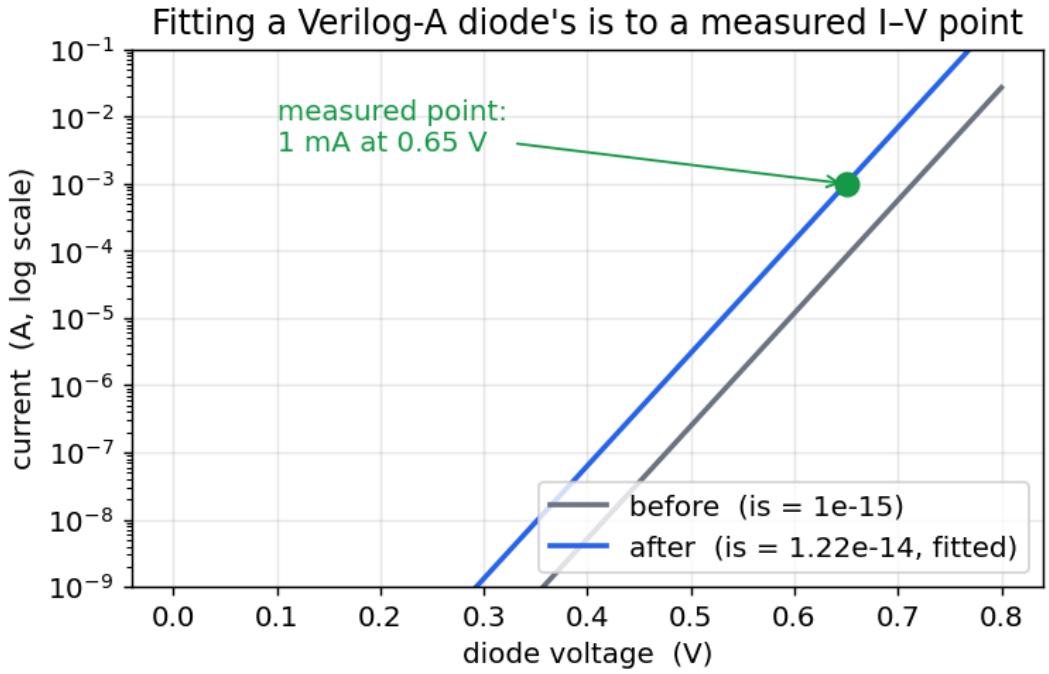
The more useful OSDI task is model parameter extraction — you have measured data for a device and want the model parameters that reproduce it. Here is a Verilog-A diode (the Shockley equation) with its saturation current `is` and emission coefficient `n` as instance parameters:

```
// optdiode.va
`include "disciplines.vams"
module optdiode(a, c);
    inout a, c;
    electrical a, c;
    (*type="instance"*) parameter real is = 1e-14 from (0:inf);
    (*type="instance"*) parameter real n  = 1.0   from (0:inf);
    analog
        I(a, c) <+ is * (limexp(V(a, c) / (n * $vt)) - 1.0);
endmodule
```

Suppose a measurement says the diode passes 1 mA at 0.65 V, and we want the `is` that reproduces it. We bias the diode at 0.65 V, run an operating point, and drive the current toward 1 mA:

```
Diode parameter extraction: fit is so I(0.65 V) = 1 mA
Vd a 0 dc 0.65
N1 a 0 dmod
.model dmod optdiode is=1e-15 n=1

.control
pre_osdi optdiode.osdi
optimize -param @n1[is] 1e-15 1e-16 1e-12 -analysis op -minimize (abs(i(vd))-1m)^2
 ↪  -tol 1e-24
.endc
.end
```

`abs(i(vd))` is the current through the diode (what the source `Vd` must supply). The optimizer reports `@n1[is] = 1.22e-14`, and the diode now passes exactly 1 mA at 0.65 V. Plotting the whole I–V curve before and after shows the fitted curve sliding onto the measured point:



That is the everyday modeling loop — measure a device, write a Verilog-A model, and let `optimize` recover the parameters — done entirely inside ngspice. You fit several parameters at once by adding more `-param` flags — and, for several measured points, the dedicated least-squares mode in the next section is the better tool.

8. Fitting to measurements — the least-squares mode

Everything so far minimized one number you wrote by hand (`-minimize <expr>`). But the most common job — *fitting* a circuit or a device model to a set of measurements — is really "make all of these measurements match at once." You *can* fold them into one expression by hand (`(X-t1)^2 + (Y-t2)^2 + …`), but that is tedious and throws away useful structure. So `optimize` has a purpose-built mode for it.

Instead of `-minimize`, list each measurement as a **`-target`**:

```
-target <expression> <desired-value> [<weight>]
```

The optimizer forms the *residual* `weight·(expression − desired-value)` for each one and drives the sum of their squares to zero — a classic *least-squares* fit. You can give up to 128 targets.

Targets can come from different analyses

Each `-analysis` opens a stage, and every `-target` after it is measured on that stage's results. So a single fit can span several analyses at once — for example a DC operating point *and* an AC response. Here we fit a series `R1` and a shunt `R2` `C` so that the DC gain is 0.4 and the gain at 2 kHz is 0.221:

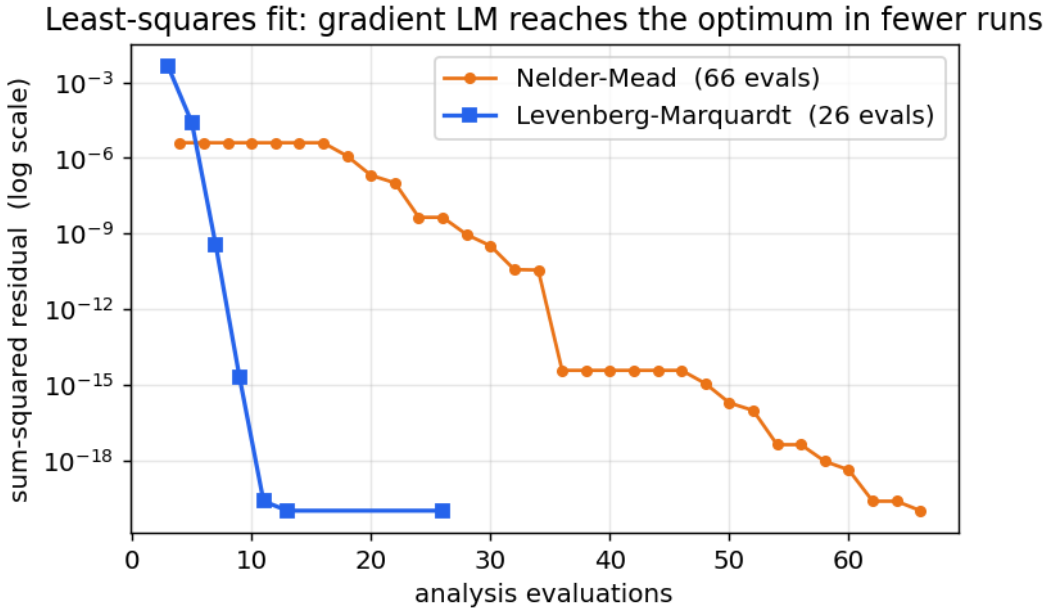
```
Least-squares fit across a DC and an AC analysis
V1 in 0 dc 1 ac 1
R1 in out 3.3k
R2 out 0 3.3k
C1 out 0 100n

.control
optimize -param R1 3.3k 500 8k -param R2 3.3k 500 8k
+        -analysis op                -target v(out)      0.4
+        -analysis ac lin 1 2000 2000 -target mag(v(out)) 0.221061
```

```
.endc
.end
```

The optimizer reports `R1 = 3 kΩ`, `R2 = 2 kΩ` — the one circuit that satisfies both goals — recovered from a 3.3 k / 3.3 k start.

It fits faster, too

When you write the objective as a list of residuals, the optimizer knows it is a least-squares problem and switches to the Levenberg-Marquardt method — a gradient-based search that estimates the slope of each residual (a *Jacobian*) and steps straight toward the bottom, instead of the slower "shrinking triangle" of Nelder-Mead. On smooth problems this is dramatic: the same two-target filter fit below reaches the optimum in 26 evaluations with Levenberg-Marquardt versus 66 with Nelder-Mead.



You can force either method with **`-method nm`** (Nelder-Mead) or **`-method lm`** (Levenberg-Marquardt); by default a `-target` fit uses `lm` and a scalar `-minimize` uses `nm`.

Device parameter extraction, the proper way

The diode fit from the previous section had just one measured point. With two points we can recover both `is` and `n` at once. Measure the diode's current at two voltages (here 0.6 V and 0.7 V), then fit both parameters as one least-squares problem — using the optional weight `1/current` so each point counts *relatively* even though the two currents differ by more than a decade:

```
Diode extraction: recover is AND n from two I-V points
Vd a 0 dc 0.6
N1 a 0 dmod
.model dmod optdiode is=5e-15 n=1.0

.control
pre_osdi optdiode.osdi
optimize -param @n1[is] 5e-15 1e-15 5e-14 -param @n1[n] 1.0 0.5 2.0
+        -analysis dc Vd 0.6 0.7 0.1
+        -target abs(i(vd))[0] 2.4856e-6  402315
+        -target abs(i(vd))[1] 6.2326e-5  16045
```

```
.endc
.end
```

`abs(i(vd))[0]` and `[1]` index the two points of the DC sweep. From a deliberately wrong start (`is = 5e-15`, `n = 1.0`) the fit recovers `is = 1e-14`, `n = 1.2` — the exact values the "measurements" came from.

> A note on targeting a single point. Like `-minimize`, each `-target` reads the last value of its expression. Use a one-point analysis (e.g. `ac lin 1 f f`) or a vector index (`v(out)[3]`, as above) to pin a specific point.

9. Knobs that aren't device instances (**`-mparam`**, **`-dparam`**)

So far every knob has been a device instance — `R1`, `C1`, or an instance parameter like `@m1[w]` — which `optimize` changes on the spot with `alter`. Two other kinds of knob need a different mechanism, and each has its own flag.

Model-card parameters — **`-mparam`**

A `.model` card holds parameters shared by every device that uses it — a diode model's `is`, a transistor model's `vth0`, a Verilog-A resistor model's `r`. These are not device instances, so `alter` (and `.dc`) can't reach them; ngspice changes them with a different command, `altermod`. Use **`-mparam`**, and name the knob `@<model>[<param>]`:

```
Vd a 0 dc 0.65
D1 a 0 dmod
.model dmod d(is=1e-15 n=1)
.control
optimize -mparam @dmod[is] 1e-15 1e-16 1e-12 -analysis op -minimize
 ↪  (abs(i(vd))-1m)^2
.endc
```

This fits the diode model's saturation current so the diode passes 1 mA at 0.65 V (`is` → `1.22e-14`). Like `-param`, `-mparam` changes the value in place — it's just as fast, no deck re-read.

Symbolic **`.param`** values — **`-dparam`**

Netlists are also often written with symbolic parameters:

```
.param rtop=1k
R1 in out {rtop}
```

`rtop` is not a device, and `alter` can't touch it: a `.param` is worked out when the deck is first read, and then it's gone. To turn a `.param` knob the optimizer has to edit the deck and read it again. It does that for you — just use **`-dparam`** instead of `-param`:

```
.param rtop=1k
V1 in 0 dc 1
R1 in out {rtop}
R2 out 0 1k
.control
optimize -dparam rtop 1k 100 10k -analysis op -minimize (v(out)-0.3)^2
.endc
```

This tunes `rtop` until `v(out) = 0.3`, giving `rtop = 2333.3` Ω. It works for a `.param` used inside an expression too — `R2 out 0 {1k*kdiv}` and `-dparam kdiv …` is fine.

Everything else is the same — `-dparam` obeys the same `init lo hi`, works with `-minimize` or `-target`, and all three kinds mix in one command (some knobs instances, some model params, some symbolic). A couple of things worth knowing:

- **`-dparam` is slower per step.** `-param` and `-mparam` change the live circuit instantly; `-dparam` re-reads the whole deck each time. So reach for `-param` (instance) or `-mparam` (model) when you can, and use `-dparam` only for genuine `.param`s.
- **It's quiet.** The re-read normally prints a `Reset re-loads circuit …` line; during optimization those are suppressed, so you don't see hundreds of them.

Which flag? `-param` for a device or `@instance[param]`; `-mparam` for a `@model[param]`; `-dparam` for a `.param`. If ngspice says your `-param` knob is "not in the circuit," it's probably a model parameter (use `-mparam`) or a `.param` (use `-dparam`).

10. Writing a good cost expression

The cost is the only tricky part, and the recipe is simple: make it zero when the circuit is perfect, and positive otherwise. Some patterns:

Goal	`-minimize` expression
make `X` equal a target `t`	`(X - t)^2`
make `X` *as large as possible*	`-X` (minimizing −`X` maximizes `X`)
make `X` *as small as possible*	`X` (or `X^2` if `X` can be negative)
satisfy two goals at once	add them: `(X-t1)^2 + (Y-t2)^2`
weight one goal more	scale it: `(X-t1)^2 + 100*(Y-t2)^2`

`X` and `Y` are any ngspice expressions over the results — node voltages `v(out)`, branch currents `i(v1)`, magnitudes `mag(v(out))`, decibels `db(v(out))`, and so on. If the expression produces a whole waveform (as in a transient), the optimizer uses its last value — the value at the end of the run.

---

11. Tips and common pitfalls

- **Give sensible bounds.** `lo` and `hi` define the search box; pick a range you know contains a good answer. The starting value `init` should be inside it.
- **Different scales are fine.** You can optimize a kilohm resistor and a nanofarad capacitor together — the optimizer works in a normalized 0-to-1 version of each range internally, so no parameter dominates just because its numbers are bigger.
- **Fast analyses optimize fastest.** The optimizer runs your `-analysis` dozens to a few hundred times, so a heavy transient makes the whole thing slow. Use the lightest analysis that captures your goal (an `op` or a one-point `ac` if you can).
- **Local minima.** Like a hiker in fog, the search can settle into a nearby dip that isn't the deepest valley. If the result looks wrong, try a different starting value or tighter bounds.
- **It's silent on purpose.** No per-iteration output means the optimization succeeded quietly; add `-verbose` to see the cost each step.
- **The circuit is left at the optimum.** After `optimize` returns, the winning values are already applied, so `print`, `plot`, `wrdata`, etc. all see the optimized circuit.

---

12. Escaping local minima — the global methods, and large problems

The methods so far (`nm`, `lm`) walk downhill from your starting guess, so they find the *nearest* valley. When the best fit is unique — a well-posed extraction or filter design — that valley is the answer, and these

local methods are fast and exact. But some cost surfaces are bumpy, with several valleys of different depth; a downhill walk then lands in whichever one it started above, which may not be the deepest. For those, `optimize` offers three global methods that *explore* the whole allowed range before settling:

`-method`	Idea	Character
`pso`	particle swarm — a flock of trial points, each pulled toward its own best and the swarm's best, carrying momentum	sweeps the space, then converges together
`de`	differential evolution — a population where each new trial is built from the *difference* of two random members	the step self-scales to the population's spread; robust on rugged, poorly-scaled surfaces
`sa`	simulated annealing — a single walker that sometimes accepts an *uphill* move (with probability `exp(-cost_rise / T)`), then "cools"	lightest — one run per step; climbs out while hot, settles as it cools

On the deliberately bumpy `f(p) = sin(p) + sin(10 p / 3)` over `[2.7, 7.5]` (best at `p = 5.146`), started at the trapping `p = 2.7` corner, Nelder-Mead sticks in a shallower dip (`-1.20`) while all three global methods reach the true best (`-1.90`). A fixed `-seed` makes any global run exactly repeatable; `-swarmsize` sets the population for `pso`/`de`. Runnable sets: `psoopt_examples`, `deopt_examples`, `saopt_examples`.

Larger problems

A run may declare up to 128 knobs and 128 targets. For a well-posed fit — one with a single clear best — Levenberg-Marquardt (`lm`) is the right tool even at that size: it solves a 100-parameter least-squares to machine precision in about two seconds. The global methods also *function* at that scale (with an adaptive high-dimensional recipe for `de`), but global search in high dimension is intrinsically expensive, so reserve it for problems that are genuinely multimodal. In short:

Your problem	Reach for
A unique best fit / extraction (even with many knobs)	`lm` — gradient least-squares, machine precision
Smooth and unimodal, but no good gradient	`nm` — the downhill simplex
Bumpy, many local minima	`pso` / `de` / `sa` — global search

Runnable set: `opt100_examples` fits 100 resistors to 100 node-voltage targets at once.

12b. Multi-objective and margin-driven runs

`nsga2` (E-216) is the multi-objective member of the family: instead of collapsing several goals into one weighted cost, it evolves a population toward the Pareto front and returns the whole non-dominated set. Use it when the trade-off itself is the answer and any scalarising weight would be arbitrary.

`wcd` (E-305) is not an optimizer in the same sense but belongs in the same workflow: it searches for the most-probable failure point and reports its distance from nominal in sigma. A design that optimises well nominally but sits 1.8σ from a spec edge is usually the wrong answer, and `wcd` is what makes that visible.

13. How it works, briefly

Under the hood, `optimize` uses the Nelder–Mead downhill-simplex method — a classic derivative-free optimizer. For `N` knobs it keeps `N+1` trial points (a "simplex"), and each step reflects the worst point through the others, expanding when that helps and contracting when it doesn't, so the shape tumbles and shrinks downhill until it settles on the minimum. It needs no derivatives — only the ability to run the circuit and read a number — which is exactly what a SPICE simulation gives it.

For every trial it applies the candidate values — instance knobs (`-param`) in place with `alter`, model-card knobs (`-mparam`) in place with `altermod`, and symbolic `.param` knobs (`-dparam`) by rewriting the deck with `alterparam` and re-sourcing it — runs your analysis, and evaluates your cost expression. It searches in a normalized `[0, 1]` version of each parameter's range so that very different component scales are treated evenly.

When you give `-targets` instead of `-minimize`, the objective is a sum of squared residuals, and `optimize` switches (by default) to Levenberg–Marquardt: it estimates the slope of each residual with a finite difference (a *Jacobian*), forms the normal equations, and solves a small damped linear system for the next step — decreasing the damping when a step succeeds and increasing it until one does. Exploiting the least-squares structure this way reaches the optimum in far fewer circuit evaluations than the simplex on smooth problems (§8).

The global methods (`pso`/`de`/`sa`, §12) reuse the very same machinery — they search the same normalized `[0, 1]` box and score every candidate through the same cost path — so they work with `-minimize` and `-target`, and with every knob kind, exactly like the local methods; they only differ in *how* they propose the next candidates.

The implementation lives in `ngspice-46/src/frontend/com_optimize.c`; the design notes are in Enhancement-130, 143, 144, 145 (least-squares and knob kinds), 194, 195, 196 (the global methods) and 197 (128-knob problems), and a runnable example set is under `examples/optimize_examples/`.